

Use Sales_Analytics Data base

SELECT + DISTINCT (Used to fetch unique values)

1. List all distinct cities where customers live.
 2. Retrieve distinct product categories from the Products table.
-

SELECT + ALIAS (AS) (Used to rename columns or expressions)

3. Display customer names and their email IDs renamed as Customer_Name and Email_ID.
 4. Show product name, price, and (Price $\times$ 2) as DoublePrice.
 5. Show product name and price after adding a 10% tax.
-

WHERE CLAUSE WITH OPERATORS

6. Find all customers who live in Hyderabad.
7. Show all products priced above 10,000.
8. List all orders placed after 2025-01-12.
9. Find products whose price is NOT equal to 1500.
10. Find customers whose Email is NULL.
11. List all orders where quantity is NOT NULL.
12. List the female customers who live in Chennai.
13. Display customers who live in ('Chennai', 'Bangalore', 'Hyderabad').
14. Show products whose category is NOT IN ('Electronics', 'Furniture')

ORDER BY & LIMIT (Questions mainly about sorting and/or limiting output)

1. List all customers ordered by their Customer Name in ascending order.
 2. Display the top 3 products by price.
 3. List the top 3 most expensive products whose price is $> 5,000$, ordered by price DESC.
 4. Show the number of customers in each city and order the result by customer count DESC.
 5. Retrieve customers in ('Chennai', 'Pune', 'Hyderabad') and sort by name.
 6. Retrieve customers in given cities, display City + CustomerName, sorted by city and name.
 7. List customers whose name starts with 'A' and sort by CustomerID.
-

AGGREGATE FUNCTIONS (Direct aggregate without GROUP BY)

8. Find the total number of customers in the Customers table.

GROUP BY AND HAVING

9. Show the number of customers in each city.
10. Show the total number of customers in each gender group.
11. Display City and Customer Name grouped by city.
12. Show only those cities where customer count is greater than 2.
13. Total orders per customer, but show only customers with more than 3 orders.
14. Product ID and total quantity sold, but only products where total quantity is between 5 and
15. Each category and its average price, but only categories where average price is greater than 5000.

JOIN-BASED QUESTIONS

1. List all orders along with the customer names.
 2. Show all customers and their orders using LEFT JOIN.
 3. Show salesperson name and orders assigned to them (RIGHT JOIN).
 4. List all orders with Customer Name, Product Name, Quantity, and Order Date (ordered by OrderID).
 5. Display all orders from 'Chennai' customers with the product details.
 6. Display all customers who purchased "Laptop".
-

BUILT-IN FUNCTIONS + AGGREGATION

7. Show total sales amount (Price $\times$ Quantity) for each order.
8. Find the top 5 customers by total purchase amount.
9. Show each Salesperson's region and the total sales value.
10. Find the most sold product with total quantity.
11. Show the earliest (minimum) and latest (maximum) order date.

DATE & TIME BUILT-IN FUNCTIONS

(Date filtering, extracting month/week/day)

12. Find all orders placed in January 2025.
13. List orders week-wise (show week number and total orders).
14. Display orders along with the day name (Monday, Tuesday, etc.).
15. Extract day name and display total sales amount and quantity per day of week (ordered Monday $\rightarrow$ Sunday).

SINGLE ROW SUBQUERIES

(returns only one value)

1. Find customers who live in the same city as **'Arjun Kumar'**.
 2. Find products that are more expensive than the **average product price**.
 3. Find products belonging to the **same category as 'Laptop'**.
 4. Find customers who have placed **more orders than the customer with CustomerID = 5**.
 5. Find products that are **priced higher than the product with ProductID = 3**.
 6. Find customers who placed an order on the **same date as OrderID = 1**.
 7. Find the salesperson whose **target amount is higher than the average target**.
-

MULTI ROW SUBQUERIES

(returns multiple values – IN, ANY, ALL)

8. Find customers who have ordered **any product in the 'Electronics' category**.
 9. Find products that were ordered by customers from **Chennai**.
 10. Find salespersons who handled orders for **customers from Bangalore**.
 11. Find products whose **price is greater than ALL products in the Furniture category**.
 12. Find products whose **price is greater than ANY Electronics product**.
 13. Find customers who purchased **products costing more than 10000**.
-

CORRELATED SUBQUERIES

(inner query depends on outer query)

14. Find customers who have placed **at least one order**.
15. Find customers who have placed **more than 2 orders**.
16. Find products that have been **ordered more than once**.
17. Find salespersons who have handled **more than 3 orders**.
18. Find customers who have purchased **products more expensive than the average product price in that category**.
19. Find products whose **price is greater than the average price of their category**.
20. Find customers whose **total orders are greater than the average number of orders placed by customers**.

BASIC CTE QUESTIONS

1. Create a CTE to calculate the **total order quantity for each customer** and display customers with quantity greater than 3.
2. Create a CTE to calculate **total sales amount for each order** (Price × Quantity) and display all orders.
3. Create a CTE to calculate the **total sales amount for each salesperson**.
4. Create a CTE to calculate the **average product price by category**.
5. Create a CTE to display **customers and their total number of orders**.
6. Create a CTE to find the **top 3 most sold products based on quantity**.
7. Create a CTE to calculate **total revenue for each product**.
8. Create a CTE to display **salesperson performance (total sales vs target amount)**.
9. Create a CTE to find **cities that have more than 2 customers**.
10. Create a CTE to calculate **total sales per day**.
11. Create a CTE to find **customers whose total purchase amount is greater than the average purchase amount of all customers**.
12. Create a CTE to calculate **category-wise total sales amount**.
13. Create a CTE to find **products whose price is higher than the average price of their category**.
14. Create a CTE to display **salesperson ranking based on total sales amount**.
15. Create a CTE to find **customers who purchased more than one different product**.

WINDOW FUNCTION

1. Display each order with the total number of orders placed by that customer using a window function.
2. Show each product with its price and the average price of all products.
3. Rank all products based on price from highest to lowest using a window function.
4. Display each order with the total quantity ordered by that salesperson.
5. Rank salespersons based on the total sales amount they generated.
6. Show product price ranking within each category using PARTITION BY.
7. Display the previous order date for each customer using LAG() function.
8. Display the next order date for each customer using LEAD() function.
9. Calculate the running total of sales amount by order date.
10. Display the top 3 most expensive products using DENSE_RANK()